SecureNet: Behavioural-Based Network Intrusion Detection System

Abstract—**Signature-based intrusion detection recognises only attacks that have already been described in its rules, and purely anomaly-based detection can flag unusual traffic without saying what kind of attack it is. This paper presents SecureNet, a behavioural network intrusion detection system that combines both ideas in a single hybrid model. An Isolation Forest assigns every network flow an anomaly score, and that score is appended to the flow's own features rather than used to filter which flows are examined further. An XGBoost classifier then assigns every flow, using both the original features and the anomaly score, to one of fifteen classes learned from the CSE-CIC-IDS2018 (CICIDS2018) dataset: Benign or one of fourteen individual attack types. Because the anomaly score is simply additional evidence for the classifier rather than a gate, every flow is classified directly, and the anomaly detector's judgement becomes part of what the classifier is allowed to use. Both models use 79 CICIDS2018 flow features. On an 80/20 train-test split of the dataset, the complete pipeline reached an overall accuracy of 95.85%. The model is deployed in a real-time application that captures packets, assembles flows, extracts the same features used in training, and raises dashboard, desktop and email alerts.**

Keywords—network intrusion detection system, behavioural anomaly detection, Isolation Forest, XGBoost, hybrid ensemble model, machine learning, CSE-CIC-IDS2018, flow-based traffic analysis, multiclass attack classification, network security, real-time monitoring, cybersecurity

I. Introduction

Network intrusion detection remains a critical line of defence against an attack surface that is probed continuously — automated scripts attempt password guessing, volumetric floods attempt to exhaust services, and compromised hosts await commands from remote command-and-control infrastructure. An intrusion detection system (IDS) inspects network traffic and raises an alert when it carries evidence of such activity. Detection strategies fall into two broad categories: misuse detection, which matches traffic against known attack signatures, and anomaly detection, which models expected behaviour and flags departures from it [1]. Signature-based detection is precise for attacks already catalogued in its rule base, but a lightly modified or previously unseen technique evades it, and its rules require continual manual maintenance as new threats emerge.

Behavioural detection approaches the problem differently: rather than asking whether a flow matches a known pattern, it asks whether the flow resembles the traffic that normally traverses the network. This allows previously unseen activity to be flagged even without a matching signature, but a deviation score alone provides weak operational evidence — being able to say only that a flow is anomalous still does not determine whether it reflects a denial-of-service (DoS) attack, a brute-force login attempt, or a benign event that happens to be statistically rare. Supervised classifiers trained on labelled flows supply the missing attack category, but they recognise only the classes present in their training data and offer no measure of how far a flow departs from normal behaviour.

Public flow-based datasets such as CICIDS2018, constructed using the methodology of Sharafaldin et al. [2], make it practical to study both strategies on common ground. Most published work on this dataset trains and evaluates a single supervised classifier offline. Comparatively few studies combine an unsupervised anomaly screen with a supervised classifier, and fewer still deploy such a model in a running system that captures live packets and computes the same features used during training.

This paper presents SecureNet, a behavioural intrusion detection system built around a single hybrid model. An Isolation Forest [3] scores every flow for how anomalous it is, and this score is appended to the flow's own feature vector as input to an XGBoost classifier [4], which assigns every flow directly to one of fifteen classes learned from CICIDS2018, rather than being filtered by a threshold before reaching the classifier. The contributions of this work are as follows:

- a hybrid detection architecture in which an unsupervised Isolation Forest anomaly score is fused with the flow's native features as input to a single supervised XGBoost classifier, so that every flow is screened and classified in one step rather than through a sequential gate;
- an evaluation of the complete pipeline on CICIDS2018 across all fifteen attack classes, with none merged, including a per-class analysis that identifies infiltration as the primary weakness; and
- a real-time implementation that captures packets, assembles network flows, extracts the 79 CICIDS2018 features used during training, applies the trained models in batches, and raises dashboard, desktop and email alerts.

The remainder of the paper is organised as follows. Section II reviews related work. Section III describes the methodology and the system architecture. Section IV covers the implementation, Section V presents and discusses the results, and Section VI concludes the paper.

II. Related Work

A. Benchmark Datasets and Their Limitations

Sharafaldin et al. [2] proposed a profile-based method for generating intrusion datasets in which benign user behaviour and attack scenarios are scripted, executed on a testbed, and converted into labelled flow records with CICFlowMeter [5]. CSE-CIC-IDS2018, produced by the Communications Security Establishment and the Canadian Institute for Cybersecurity with the same methodology, covers benign traffic together with brute-force, DoS, DDoS, web, infiltration and botnet attacks, and it is the source of all training and testing data in the present study. Later studies have examined these datasets critically. Engelen et al. [6] traced errors in flow construction and labelling in the closely related CICIDS2017 dataset and released a corrected version. Flood et al. [7] described recurring design problems, which they called data design smells, across widely used benchmark NIDS datasets, and Goldschmidt and Chudá [8] surveyed public intrusion datasets and discussed issues such as class imbalance and labelling quality. These findings do not make CICIDS2018 unusable, but they mean that results obtained on a single split of the data should be interpreted with care.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

B. Unsupervised Anomaly Detection with Isolation Forest

Liu et al. [3] introduced Isolation Forest, which builds an ensemble of random binary trees and treats points that are separated after only a few random splits as anomalous. The method needs no labels, uses sub-sampling, and has a time complexity that is linear in the number of samples, which explains its popularity as a first-stage detector for network traffic. Moomtaheen et al. [9] applied the Extended Isolation Forest to Zeek connection logs and found that its effectiveness varied with the attack tactic, illustrating that an unsupervised score alone does not separate all attack types equally well. Marteau et al. [10] extended Isolation Forest into a hybrid form that reduces a bias of the original scoring and can use a small amount of labelled data, and they tested it on intrusion detection benchmarks. Both studies remain at the scoring stage: they decide whether a record is unusual, not which attack category it belongs to.

C. Supervised Classification on CSE-CIC-IDS2018

Gradient-boosted decision trees, and XGBoost [4] in particular, are among the most frequently reported classifiers on CICIDS2018. Chimphlee and Chimphlee [11] tuned XGBoost for CSE-CIC-IDS2018 and reported that it compared favourably with the other algorithms they evaluated. Songma et al. [12] organised their study into three phases, namely data preparation, feature reduction and classifier comparison, and found tree-based ensembles among the strongest models. Mohamed et al. [13] evaluated machine-learning classifiers on CSE-CIC-IDS2018 in binary and multiclass settings. Deep architectures have also been tested: Wang et al. [14] compared six neural models, including CNN, LSTM and hybrid designs, for benign traffic and six attack types and obtained multiclass accuracy above 98%, while noting that the hybrid models had longer inference times. Xu et al. [15] represented flows as edges of a graph and trained a graph neural network with self-supervised learning. These supervised approaches are accurate on the classes seen in training, but they act as closed-set classifiers and are usually evaluated offline rather than on captured traffic.

D. Hybrid and Real-Time Detection Systems

Several studies combine the two families. Miran and Kadir [16] compared Random Forest and XGBoost with two unsupervised detectors, Isolation Forest and One-Class SVM, using CICIDS2017; XGBoost was the strongest model for known attacks, the unsupervised models handled unlabelled behaviour, and the authors argued for a hybrid system without building one. Macko et al. [17] placed a supervised binary Random Forest filter in front of a variational-autoencoder anomaly detector on UGR'16 traffic and reported a higher AUC and about 30% more detected attacks with almost no increase in false positives. Their order is the reverse of the one used here: the classifier runs first and the anomaly detector second, whereas SecureNet screens with the unsupervised model first and lets the classifier name or cancel each alarm. Thirimanne et al. [18] built a real-time IDS around a deep neural network fed by a live feature extractor, but its detector is purely supervised. Granadillo and Kaaniche [19] evaluated machine-learning and deep-learning models for anomaly detection and traffic classification on NetFlow data.

E. Research Gap

The literature therefore offers modern flow datasets, well-understood unsupervised detectors, strong supervised classifiers on CICIDS2018, and a few hybrid or real-time designs. Less common is a system that places an unsupervised screen before a multiclass classifier on CICIDS2018, lets the classifier cancel false alarms, reports per-class results for the complete pipeline, and runs the same trained models on captured traffic with a verified match between offline and deployed behaviour. SecureNet addresses this combination.

III. Methodology and System Architecture

The system is organised as two pipelines that share one hybrid model. The training pipeline in Fig. 1 builds and evaluates the model on CICIDS2018, and the live pipeline in Fig. 2 applies the saved models to traffic captured on the monitored computer. In both pipelines, Isolation Forest generates an anomaly score for every flow, and XGBoost uses that anomaly score together with the flow's own features to decide what the flow is.

A. Dataset and Preprocessing

As Fig. 1 shows, the CICIDS2018 dataset first goes through preprocessing and feature selection. The ten daily CICIDS2018 files were merged, repeated header rows were removed, rows containing infinite, NaN or empty values were dropped, corrupt rows were removed, exact duplicate rows were removed, rows whose features appeared under more than one label were removed, data types were fixed and the rows were shuffled. Feature selection retains 79 of the CICIDS2018 flow features, which are referred to by name so that the live extractor can supply them in the same order. All fifteen original CICIDS2018 labels are kept separate, with no classes merged: Benign and fourteen attack types, including the dataset's own "Infilteration" spelling. For the Isolation Forest, the features are log-transformed, with values floored at -1 before the transform so that the dataset's "not set" sentinel value maps to zero, and then standardised with a scaler fitted on the Benign training rows only. XGBoost receives the untransformed features together with the Isolation Forest's anomaly score, since tree splits do not depend on monotonic rescaling.

The preprocessed data are then split into a Training Dataset (80%) and a Testing Dataset (20%), stratified with a fixed random seed so that the class balance is preserved in both halves. The Testing Dataset is kept aside and is not used to fit either model.

B. Stage 1: Isolation Forest Anomaly Score

Isolation Forest isolates a point by recursively choosing a random feature and a random split value between the minimum and maximum of that feature [3]. Anomalous points tend to be separated after fewer splits, so their path length from the root to a leaf is short. For a flow x in a sample of size n, the anomaly score defined by Liu et al. is

$$s(x,n)=2^{\frac{-E[h(x)]}{c(n)}} \tag{1}$$

where $h(x)$ is the path length of x in one tree, $E[h(x)]$ is its average over all trees, and $c(n)$ is the average path length of an unsuccessful search in a binary search tree with n points, used for normalisation:

$$c(n)=2H(n-1)-\frac{2(n-1)}{n},\quad H(i)\approx \ln(i)+0.5772 \tag{2}$$

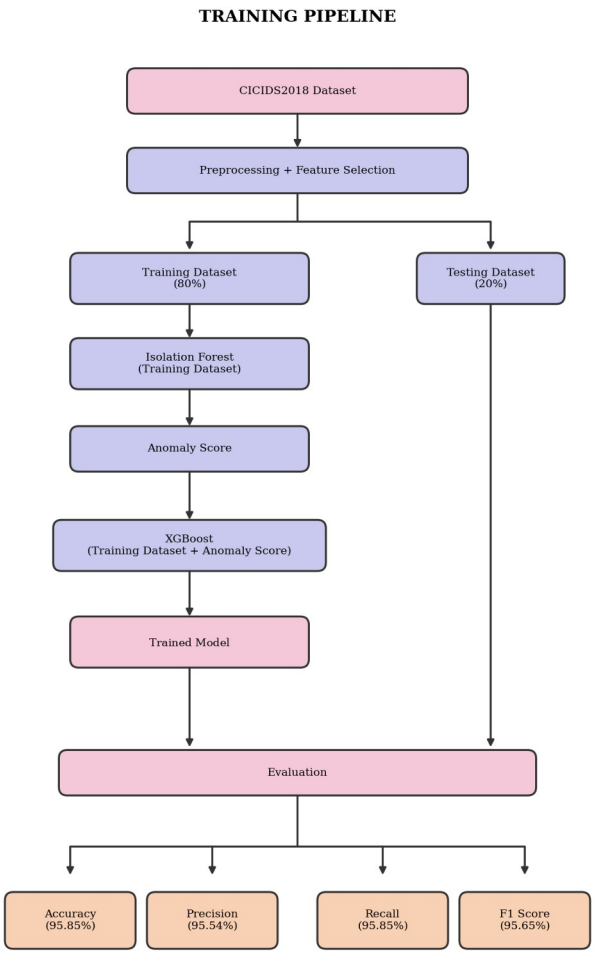


Fig. 1. Training pipeline. The CICIDS2018 dataset is preprocessed and reduced to 79 features, split into a Training Dataset (80%) and a Testing Dataset (20%). The Training Dataset is passed to the Isolation Forest, which generates an anomaly score; XGBoost takes the Training Dataset together with the anomaly score and produces the Trained Model. The Testing Dataset is brought back in at Evaluation, alongside the Trained Model.

Anomaly scores close to 1 indicate likely anomalies, and anomaly scores well below 0.5 indicate flows that resemble the bulk of the data. The scikit-learn implementation returns this anomaly score with the sign inverted, so SecureNet uses the negated output of score_samples(), which lies in (0, 1]. The forest contains 300 trees, each built on a sub-sample of 8,192 flows drawn from the Benign training rows, with a contamination parameter of 0.02. Rather than gating which flows reach the classifier, every flow's anomaly score is kept and passed forward; an informational threshold was also measured on the training split, for reference only, and is reported in Table II.

C. Stage 2: XGBoost Classification

XGBoost [4] is a gradient-boosted decision tree classifier. It takes each flow's features together with the Isolation Forest's anomaly score and predicts which of the fifteen CICIDS2018 classes the flow belongs to. The trained model contains 200 boosting rounds, with a maximum tree depth of 10 and a learning rate of 0.1, using the multi:softprob objective across the fifteen classes. Its highest class probability is also used as the classifier's confidence.

D. Score Fusion and Classification

The two stages are combined by feature-level fusion rather than a sequential gate. The Isolation Forest's anomaly score is appended to the flow's 79 features as an 80th input, and XGBoost is trained and evaluated on all 80 values together, across every class including Benign:

$$\hat{y}(x)=\begin{cases} Benign, & s(x)<\theta \\ \arg\max_{c} P(y=c/x), & s(x)\geq\theta \end{cases} \quad (5)$$

Every flow is classified directly; Benign is an ordinary class the classifier may output, not a default applied to low-anomaly-score flows, and no flow is withheld from the classifier by a threshold. Algorithm 1 summarises the procedure used for evaluation and in the live system.

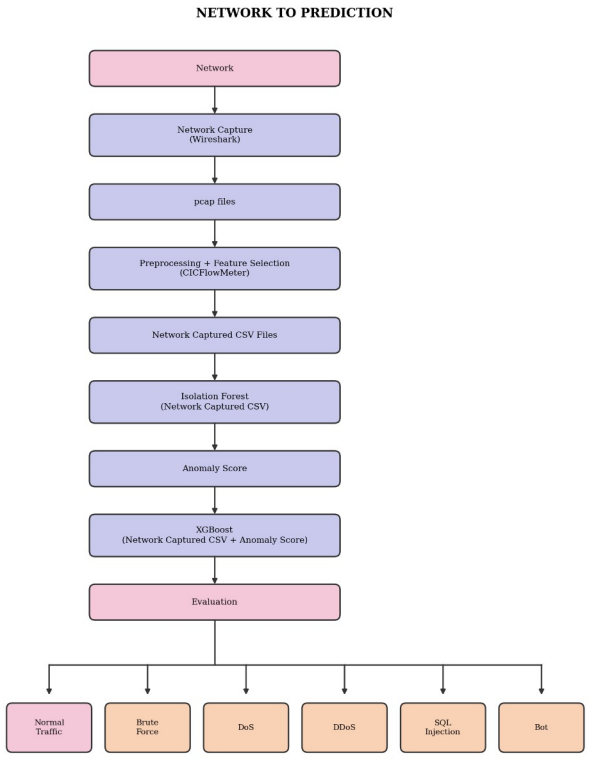


Fig. 2. Live prediction pipeline. Network traffic is captured with Wireshark into pcap files, which go through preprocessing and feature selection (CICFlowMeter) into Network Captured CSV Files. Isolation Forest scores every row of the Network Captured CSV, producing an Anomaly Score; XGBoost takes the Network Captured CSV together with the Anomaly Score and is evaluated to produce the prediction: Normal Traffic, Brute Force, DoS, DDoS, SQL Injection, Bot, or one of the other CICIDS2018 attack types.

Algorithm 1: SecureNet hybrid detection

Input: captured network traffic; models S (scaler), IF, XGB
Output: alert with predicted label $\hat{y}$ and anomaly score s

for each incoming flow **do**
  $\mathbf{x} \leftarrow$ Capture(network) extract flow features
  $\mathbf{z} \leftarrow S(\mathbf{x})$ scale features
  $s \leftarrow IF$.score($\mathbf{z}$) anomaly score
  $\mathbf{x}' \leftarrow$ concat($\mathbf{x}$, s) append anomaly score
  $\mathbf{p} \leftarrow XGB$.predict_proba($\mathbf{x}'$) class probabilities
  $\hat{y} \leftarrow$ argmax($\mathbf{p}$) predicted class
  if $\hat{y} \neq$ Benign **then**
    Alert($\hat{y}$, s) report attack
  end if
end for

E. Live Prediction Pipeline

Fig. 2 shows how the saved models are applied to real traffic. Network traffic is captured with Wireshark and written out as pcap files. These go through the same preprocessing and feature selection step as training (CICFlowMeter), producing Network Captured CSV Files with the same feature names and the same time units as the training data. The Isolation Forest scores every row of this CSV, producing an anomaly score; the anomaly score is appended to the flow's features, and XGBoost classifies the result. The predicted class is reported as Normal traffic or as abnormal traffic with the predicted attack type.

F. Evaluation Metrics

The complete pipeline is evaluated on the Testing DatasetTP_c, FP_c and FN_c denoting the true positives, false positives and false negatives of class c, C the number of classes and N the number of test flows, accuracy is

$$Accuracy = \frac{1}{N}\sum_{c=1}^{C} TP_c \tag{6}$$

Precision, recall and F1-score are computed for each class,

$$Precision_c = \frac{TP_c}{TP_c + FP_c}, Recall_c = \frac{TP_c}{TP_c + FN_c} \tag{7}$$

$$F1_c = \frac{2 \cdot Precision_c \cdot Recall_c}{Precision_c + Recall_c} \tag{8}$$

These are summarised with macro averaging, which weights all classes equally, and weighted averaging, which weights each class by its number of test samples.

The separation achieved by the stage-1 anomaly score is also summarised by the area under the ROC curve (ROC-AUC).

IV. Implementation

SecureNet is implemented in Python for detection and the server, and in TypeScript for the dashboard. Table I lists the technology used in each layer. The models were trained with scikit-learn 1.9.1 and XGBoost 3.4.1, and the same versions are installed in the application so that the saved models behave exactly as they did during evaluation.

TABLE I. Technology Stack

Layer	Technology
Training dataset	CSE-CIC-IDS2018 (79 features, 15 classes)
Detection models	Isolation Forest (scikit-learn 1.9.1), XGBoost 3.4.1
Packet capture	Wireshark (Windows 10/11)
Backend	FastAPI
Database	PostgreSQL (cloud-hosted)
Live updates	WebSocket stream to the dashboard
Dashboard	React
Alerts	Dashboard pop-ups, Windows notifications, SMTP email

A. Model Files and Parameters

The trained pipeline is stored as five files: the fitted scaler (scaler.joblib), the Isolation Forest (isolation_forest.joblib), the XGBoost booster in its native JSON format (xgboost_model.json), a configuration file listing the 79 feature names, the class names and the informational threshold (if_config.json), and the test results (metrics.json). Table II lists the model parameters. The maximum depth and learning rate are not stored in an XGBoost model file and must be taken from the training script.

TABLE II. Model Parameters

Parameter	Value
Isolation Forest: number of trees	300
Isolation Forest: samples per tree	8,192
Informational anomaly threshold (not a gate)	0.5
XGBoost: boosting rounds	200
XGBoost: number of classes	15
XGBoost: max depth / learning rate	10 / 0.1
Input features	79 (+1 anomaly score)

B. Live Detection Service

The capture thread hands every finished flow to an asynchronous queue, and a worker takes up to 256 waiting flows at a time and scores them in one call, which gives the same verdicts as scoring them one by one. To keep the monitor stable under flooding attacks, at most 50,000 flows are kept open and at most 20,000 packets are counted per flow, and alert storage is limited to 120 incidents per minute (30 per source), with the remaining flows still counted. A whitelist of trusted addresses, ports and protocols is applied before detection.

Severity follows the classifier's confidence: high at 0.9 or above, medium at 0.6 or above, and low otherwise.

Alerts are pushed to the dashboard over a WebSocket and shown as pop-ups and Windows notifications. Registered users also receive an email with the incident report attached: the first email is sent when the network turns from normal to abnormal, and while the attack continues at most one summary is sent every five minutes. Incidents are stored in PostgreSQL, and the dashboard provides a searchable history and one downloadable report per day. The experiments were run on [PLACEHOLDER: processor, RAM and Windows version of the computer used].

V. Results and Discussion

A. Overall Performance

Table III summarises the performance of the complete hybrid pipeline on the 465,837 test flows. The pipeline classified 95.85% of the flows correctly. The stage-1 anomaly score reached a ROC-AUC of 0.895 for benign versus attack flows, measured on the training split before the score was stacked into the classifier's input.

TABLE III. PERFORMANCE OF THE HYBRID PIPELINE ON THE TESTING DATASET

Metric	Value
Test flows	465,837
Accuracy	95.85%
ROC-AUC (stage 1)	0.895
Macro F1-score	78.61%
Weighted F1-score	95.65%

B. Per-Class Results

Table IV gives the per-class results across all fifteen classes, with none merged. Benign traffic has a recall of 96.3%, which means that 3.7% of benign test flows were misclassified as an attack. The large, repetitive attack classes are detected very well: SSH-Bruteforce and DDOS attack-HOIC both reach 100% recall, DoS attacks-Hulk 100%, Bot 99.99%, Brute Force-XSS 100%, DoS attacks-GoldenEye 99.96%, DoS attacks-Slowloris 99.90%, DDOS attack-LOIC-UDP 95.66% and Brute Force-Web 93.58%.

TABLE IV. PER-CLASS RESULTS OF THE HYBRID PIPELINE

Class	Precision	Recall	F1	Support
Benign	94.19%	96.32%	95.24%	199,922
DDoS attacks-LOIC-HTTP	99.99%	99.96%	99.97%	115,073
DDOS attack-HOIC	100.00%	100.00%	100.00%	39,772
DoS attacks-Hulk	99.99%	100.00%	99.99%	29,040
Bot	100.00%	99.99%	99.99%	28,907
Infiltration	61.62%	49.50%	54.90%	23,520
SSH-Bruteforce	100.00%	100.00%	100.00%	18,808
DoS attacks-GoldenEye	99.98%	99.96%	99.97%	8,281
DoS attacks-Slowloris	99.95%	99.90%	99.92%	1,982
DDOS attack-LOIC-UDP	86.88%	95.66%	91.06%	346
Brute Force-Web	50.00%	93.58%	65.18%	109
Brute Force-XSS	95.74%	100.00%	97.83%	45
SQL Injection	80.00%	53.33%	64.00%	15
DoS attacks-SlowHTTPTest	11.11%	11.11%	11.11%	9
FTP-BruteForce	0.00%	0.00%	0.00%	8

Infiltration is the clear weak point, with only 49.5% of infiltration flows recognised. In CICIDS2018 this class is generated from a host inside the victim network after it has been compromised, and much of its traffic consists of ordinary connections, such as internal scans and downloads, that differ little from benign flows. Because the anomaly score is only one of eighty inputs XGBoost sees rather than a gate, infiltration flows do reach the classifier, but their flow-level features alone are often not distinctive enough for XGBoost to separate them from Benign. DoS attacks-SlowHTTPTest and FTP-BruteForce score lowest of all, but with only 9 and 8 test rows respectively these figures are not statistically reliable and mainly reflect the extreme rarity of those classes in CICIDS2018. Improving infiltration detection would likely require features that describe host behaviour over time rather than single flows.

C. Agreement Between Offline and Deployed Models

D. Live Monitoring

SecureNet was also run on live traffic on a Windows computer with Npcap installed.

E. Comparison and Limitations

Published accuracies on CICIDS2018 are not directly comparable with the figures reported here. Studies differ in how they clean the data, which attack labels they keep or merge, how they split the data, and how they average the metrics; for example, Wang et al. [14] report multiclass accuracy above 98% for benign traffic and six attack categories with deep models, but under a different class definition and preprocessing.

Several limitations remain. First, the results come from one train–test split of a dataset with known quality issues [6]–[8]. Second, traffic on a real local network differs from the 2018 testbed, so accuracy on live traffic can be lower than the offline 95.85%. Third, infiltration and other low-and-slow attacks that resemble normal traffic are poorly detected, since their flow-level statistics and anomaly score give XGBoost little to separate them from Benign. Fourth, XGBoost is a closed-set classifier: an attack type absent from training must still be assigned to one of the fifteen known classes, or reported as unclassified when its confidence is low, since every flow is classified rather than only the ones an anomaly gate would have flagged.

VI. CONCLUSION

This paper presented SecureNet, a behavioural network intrusion detection system built on a single hybrid model. An Isolation Forest scores every flow for unusual behaviour, and that score is appended to the flow's own features as input to an XGBoost classifier, which assigns every flow directly to one of fifteen CICIDS2018 classes. Trained on 79 CICIDS2018 features, the complete pipeline reached 95.85% accuracy on an 80/20 held-out split, with recall between 93.6% and 100% for the large, repetitive attack classes (SSH-Bruteforce, DDOS attack-HOIC, DoS attacks-Hulk, Bot, Brute Force-XSS, DoS attacks-GoldenEye, DoS attacks-Slowloris, DDOS attack-LOIC-UDP and Brute Force-Web) and 96.3% for benign traffic, while infiltration remained difficult at 49.5%. The same models run in a real-time system that captures packets, extracts matching features, and alerts users through a dashboard, desktop notifications and email.

Future work will measure detection accuracy on live attacks in a controlled network, verify that the deployed code reproduces this hybrid model's offline results the way an earlier, threshold-gated version was verified, compare the stacked design with XGBoost alone on the same split, improve infiltration detection with features that describe host behaviour over longer periods, and evaluate the model on corrected or newer datasets.

REFERENCES

[1] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1153–1176, 2016.

[2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generatin a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Funchal, Portugal, 2018, pp. 108–116.

[3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.

[5] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Porto, Portugal, 2017, pp. 253–262.

[6] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, 2021, pp. 7–12.

[7] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark NIDS datasets," in *Proc. 9th IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, Vienna, Austria, 2024.

[8] P. Goldschmidt and D. Chudá, “Network intrusion datasets: A survey, limitations, and recommendations,” arXiv:2502.06688, 2025.

[9] F. Moomtaheen, S. S. Bagui, S. C. Bagui, and D. Mink, “Extended isolation forest for intrusion detection in Zeek data,” *Information*, vol. 15, no. 7, art. 404, 2024.

[10] P.-F. Marteau, S. Soheily-Khah, and N. Béchet, “Hybrid isolation forest—Application to intrusion detection,” arXiv:1705.03800, 2017.

[11] W. Chimphlee and S. Chimphlee, “Hyperparameters optimization XGBoost for network intrusion detection using CSE-CIC-IDS 2018 dataset,” *IAES Int. J. Artif. Intell.*, vol. 13, no. 1, pp. 817–826, 2024.

[12] S. Songma, T. Sathuphan, and T. Pamutha, “Optimizing intrusion detection systems in three phases on the CSE-CIC-IDS-2018 dataset,” *Computers*, vol. 12, no. 12, art. 245, 2023.

[13] H. Mohamed, C. Sabty, and A. El Bolock, “IntrusionHunter: Detection of cyber threats in big data,” in *Proc. 12th Int. Conf. Data Sci., Technol. Appl. (DATA)*, 2023, pp. 311–318.

[14] Y.-C. Wang, Y.-C. Houng, H.-X. Chen, and S.-M. Tseng, “Network anomaly intrusion detection based on deep learning approach,” *Sensors*, vol. 23, no. 4, art. 2171, 2023.

[15] R. Xu *et al.*, “Applying self-supervised learning to network intrusion detection for network flows with graph neural network,” arXiv:2403.01501, 2024.

[16] A. Z. Miran and G. S. Kadir, “A hybrid machine learning approach for enhancing intrusion detection systems using CICIDS2017 dataset,” *Dasinya J. Eng. Informat.*, vol. 2, no. 2, 2026, doi: 10.65542/djei.v2i2.49.

[17] D. Macko, P. Goldschmidt, P. Pištek, and D. Chudá, “Assessing the impact of a supervised classification filter on flow-based hybrid network anomaly detection,” arXiv:2310.06656, 2023.

[18] S. P. Thirimanne, L. Jayawardana, L. Yasakethu, P. Liyanaarachchi, and C. Hewage, “Deep neural network based real-time intrusion detection system,” *SN Comput. Sci.*, vol. 3, art. 145, 2022.

[19] G. G. Granadillo and N. Kaaniche, “AI-based anomaly detection and classification of traffic using NetFlow,” in *Proc. 22nd Int. Conf. Secur. Cryptogr. (SECRYPT)*, 2025.

[20] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.